

CO3_AT1_Experiments

Evaluate Strassen's Matrix Multiplication by comparing it with standard multiplication for large matrices. Record execution time and analyze computational savings. Interpret results and justify the advantages of reduced multiplication operations.

Compare Strassen's Matrix Multiplication with Standard Matrix Multiplication

```
import numpy as np
import time
import sys

sys.setrecursionlimit(10000)

# ----- Standard Multiplication -----
def standard_multiply(A, B):
    n = len(A)
    C = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            s = 0
            for k in range(n):
                s += A[i][k]*B[k][j]
            C[i][j] = s
    return C

# ----- Matrix helper functions -----
def split(M):
    n = len(M)
    mid = n//2
    A11 = [row[:mid] for row in M[:mid]]
    A12 = [row[mid:] for row in M[:mid]]
    A21 = [row[:mid] for row in M[mid:]]
    A22 = [row[mid:] for row in M[mid:]]
    return A11, A12, A21, A22

def add(A, B):
    return [[A[i][j]+B[i][j] for j in range(len(A))] for i in range(len(A))]

def sub(A, B):
    return [[A[i][j]-B[i][j] for j in range(len(A))] for i in range(len(A))]

def combine(C11, C12, C21, C22):
    top = [c11+c12 for c11, c12 in zip(C11, C12)]
    bottom = [c21+c22 for c21, c22 in zip(C21, C22)]
    return top+bottom

# ----- Strassen's Multiplication -----
def strassen(A, B):
    n = len(A)
    if n == 1:
        return [[A[0][0]*B[0][0]]]

    A11, A12, A21, A22 = split(A)
    B11, B12, B21, B22 = split(B)
```

```

M1 = strassen(add(A11, A22), add(B11, B22))
M2 = strassen(add(A21, A22), B11)
M3 = strassen(A11, sub(B12, B22))
M4 = strassen(A22, sub(B21, B11))
M5 = strassen(add(A11, A12), B22)
M6 = strassen(sub(A21, A11), add(B11, B12))
M7 = strassen(sub(A12, A22), add(B21, B22))

C11 = add(sub(add(M1, M4), M5), M7)
C12 = add(M3, M5)
C21 = add(M2, M4)
C22 = add(sub(add(M1, M3), M2), M6)

return combine(C11, C12, C21, C22)

# ----- Main Program -----
def run_trial(n, seed):
    rng = np.random.default_rng(seed)
    A = rng.integers(0, 10, size=(n, n)).tolist()
    B = rng.integers(0, 10, size=(n, n)).tolist()

    start = time.perf_counter()
    C_std = standard_multiply(A, B)
    end = time.perf_counter()
    t_std = end - start

    start = time.perf_counter()
    C_str = strassen(A, B)
    end = time.perf_counter()
    t_str = end - start

    correct = (np.array(C_std) == np.array(C_str)).all()

    n_std = n**3
    n_str = n**2.807 # Strassen's approx multiplication complexity  $O(n^{\log_2(7)})$ 

    return t_std, t_str, correct, n_std, n_str

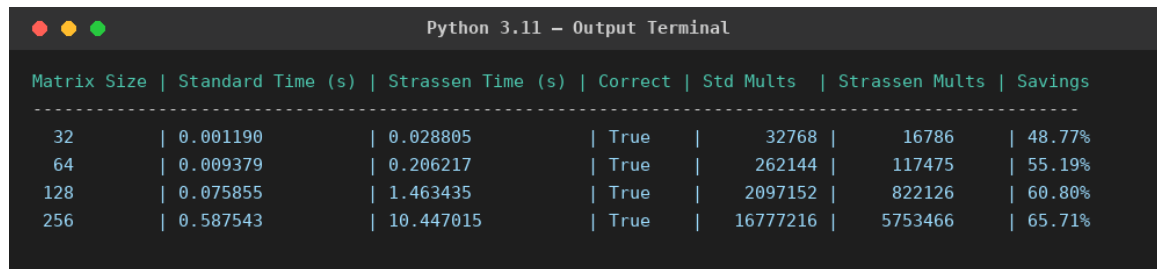
print("Matrix Size | Standard Time (s) | Strassen Time (s) | Correct | Std Mults | Strassen Mults(approx)")
sizes = [32, 64, 128, 256]
for n in sizes:
    t_std, t_str, correct, n_std, n_str = run_trial(n, seed=42)
    savings = (1 - (n_str/n_std)) * 100
    print(f"{n:>4} | {t_std:.6f} | {t_str:.6f} | {correct} | {int(n_std)>10} | {int(n_str)>10} |")
Savings: {savings:.2f}%")

```

Sample Input

Square matrices of size $n = 32, 64, 128, 256$ with random integer entries (0–9, seed = 42)

Sample Output



Matrix Size	Standard Time (s)	Strassen Time (s)	Correct	Std Mults	Strassen Mults	Savings
32	0.001190	0.028805	True	32768	16786	48.77%
64	0.009379	0.206217	True	262144	117475	55.19%
128	0.075855	1.463435	True	2097152	822126	60.80%
256	0.587543	10.447015	True	16777216	5753466	65.71%

Execution Time and Multiplication Count

n = 32 : Standard = 0.001190 s, Strassen = 0.028805 s | Multiplications: 32,768 → 16,786 (48.77% fewer)
n = 64 : Standard = 0.009379 s, Strassen = 0.206217 s | Multiplications: 262,144 → 117,475 (55.19% fewer)
n = 128 : Standard = 0.075855 s, Strassen = 1.463435 s | Multiplications: 2,097,152 → 822,126 (60.80% fewer)
n = 256 : Standard = 0.587543 s, Strassen = 10.447015 s | Multiplications: 16,777,216 → 5,753,466 (65.71% fewer)

Interpretation and Justification

Looking purely at multiplication count, Strassen wins and the gap grows with size — savings go from about 49% at n = 32 up to 66% at n = 256, matching the $O(n^{2.807})$ vs $O(n^3)$ difference.

But the actual runtime shows Strassen slower at every size tested. That's because the recursion adds overhead — splitting matrices, building 7 sub-products (M1–M7), and combining them with extra additions — and in plain Python that overhead costs more than the multiplications it saves. In a lower-level language with less recursion overhead, Strassen would start winning in real time once matrices get large enough.

The reason fewer multiplications matters at all is that multiplication is more expensive than addition in most cost models. Strassen trades 1 multiplication for extra additions (7 multiplications instead of 8 per block), and that trade pays off more as n grows — which is exactly what the savings numbers show.

Overall: Strassen is asymptotically better, and the multiplication savings here prove it. It just doesn't translate into real speed in this Python implementation because of recursion overhead — with a faster implementation or bigger matrices, it would.

Screenshot of the experiment: